Margo: Prefetcher Selection via Marginal Utility

Jacky Wong[†*], Yinting Huang[‡], Sam Ainsworth[§], Paul H. J. Kelly[†]
[†]Imperial College London, [‡]University of Cambridge, [§]IMEC
[*]Corresponding Author: jacky.wong20@imperial.ac.uk

This is a pre-release version of Margo! For final paper, please wait for the camera-ready.

***Abstract*—Modern processors employ a wide variety of hardware data prefetchers to cover diverse memory access patterns across different workloads. While effective multi-prefetcher management can significantly improve miss coverage and system performance, poor control often leads to cache pollution and exacerbates DRAM bandwidth contention.**

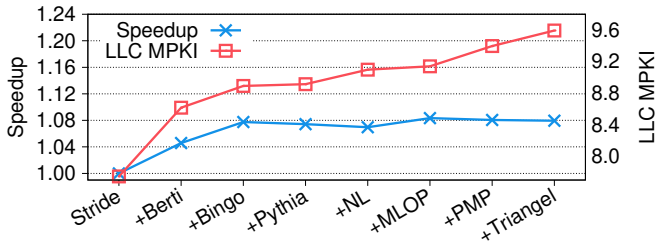
Prior prefetcher selection proposals, such as Alecto and I-POP, extensively study and quantify the performance contributions of individual prefetchers. However, they fail to account for complex interactions that occur when multiple prefetchers operate concurrently. Consequently, these schemes scale poorly as the number of managed prefetchers increases, generating prohibitive DRAM traffic without commensurate performance gain.

To overcome these limitations, we present Margo: a dynamic prefetcher management scheme that observes the fine-grained marginal effects of each managed prefetcher. We introduce *marginal utility*, a comprehensive multi-dimensional metric that combines accuracy, timeliness, and the source of prefetches to inform filtering decisions on prefetcher outputs. To support this, we introduce a novel statistical sampler and stochastic counter mechanisms that simultaneously maintain these statistics for many fine-grained, hypothetical scenarios. We also propose a bespoke input filter for temporal prefetchers which allows them to focus on accesses that no other prefetcher can cover, reducing metadata storage overhead and improving overall performance. In SPECspeed 2026, Margo outperforms the state-of-the-art prefetcher selector I-POP by 2.1% while cutting excess DRAM traffic by a factor of 3.

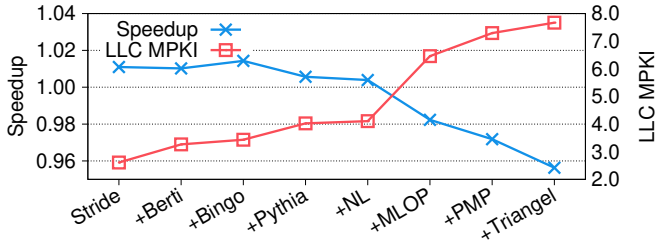
I. Introduction

Data prefetching is a primary method for bridging the increasing performance gap between processors and memory. Given the growing complexity of memory access patterns in contemporary workloads, a wide gamut of data prefetchers have emerged to cover various distinct memory behaviors like strides [13], offsets [39], [46], [47], [55], regional [5], [14], [30], [59], and irregular accesses [2], [3], [66]. Hardware vendors have thus incorporated multiple data prefetchers into their processor designs. For example, ARM's Neoverse V2 features eight different prefetchers [9]. Similarly, Nvidia's Vera and AMD's Zen 5 cores incorporate multiple data prefetchers at each cache level [1], [49]. This diversity places increasing demands on prefetch-selection mechanisms, which must arbitrate among predictions from heterogeneous sources and tolerate their inherent inaccuracies, while avoiding excessive memory traffic that would reduce energy efficiency and intensify contention in multicore systems.

The research community has made significant strides towards solving this challenge. The two state-of-the-art prefetcher selection mechanisms are Alecto [35] and I-POP [37]. Alecto [35] identifies instruction PCs that generate harmful prefetches for particular sub-prefetchers, and suppresses training for those PC–prefetcher pairs. I-POP [37] applies feedback-directed prefetching [60] to dynamically control the degree of sub-prefetchers (the number of prefetches issued per prefetcher per observation) and to disable sub-prefetchers entirely when their latency-weighted benefit no longer outweighs the traffic and cache pollution they cause. There has even been work on using LLMs [68] to statically perform prefetcher selection from offline profile and code analysis. However, we observe that state-of-the-art prefetcher selection algorithms do not scale well with the number of managed prefetchers (Fig. 1). As more prefetchers are added, speedup starts to stall, while DRAM accesses escalates rapidly. In other words, these extra prefetchers are adding no additional benefit or even degrades performance, while requesting a significant amount of DRAM traffic – even though the prefetcher selection algorithm treats the additional prefetchers as beneficial and keeps issuing their prefetches.



(a) SPECspeed 2026



(b) Cloudsuite

Fig. 1: Scalability of I-POP [37] with the number of managed prefetchers in SPECspeed 2026 and Cloudsuite. Off-chip LLC misses rise sharply with each added prefetcher without commensurate performance gain.

We identify five key limitations in existing approaches to dynamic hardware-prefetcher selection. First, prefetcher selection works best when it is filtering the outputs of prefetchers: naively filtering the inputs they see can often impair their ability to pick up patterns and evaluate their behavior [35].

Second, prefetchers should be evaluated relative to one another rather than in isolation. In particular, a more aggressive prefetcher is beneficial only when it provides value beyond that already delivered by a less aggressive alternative. Third, prefetcher selection should be fine-grained [8], [35], with different decisions for different combinations of prefetchers in different contexts (such as in different PCs). Fourth, prefetcher selection should not only consider accuracy [35], [37]. It must also account for the timeliness of both a prefetcher's own requests and those issued concurrently by other prefetchers, as well as the asymmetric benefits and costs of on-chip and off-chip prefetches [37]. Finally, when accurate, low-cost, low-state prefetchers can cover access patterns, there is no need to inform high-cost temporal prefetchers [2], [18], [66] of the same memory accesses, allowing the more expensive prefetchers to focus on more complex patterns and alleviating the pressure on shared cache-prefetcher resources.

Observing the limitations of contemporary prefetcher selectors, we introduce the concept of *marginal utility* to prefetcher management: can we measure the additional value that a prefetcher brings in the presence of other, potentially complementary, inferior or superior prefetchers? This informs the design of Margo: a bandwidth-aware prefetcher selection mechanism that is guided by the fine-grained marginal utility between different prefetchers. Margo uses a sampling mechanism to evaluate each sub-prefetcher's isolated hypothetical performance. Isolated prefetcher accuracies are represented as efficient stochastic saturating counters [53] enabling fine-grained comparison across prefetchers. By ranking the prefetchers from most to least accurate for each potentially covered prefetch, we can determine their marginal utility, i.e., how much additional latency each additional prefetcher covers. This is further improved by adding asymmetric weighting of the costs and benefits of on-chip and off-chip accesses, and a novel coverage-based filter for temporal prefetchers.

Coupled with a wide selection of state-of-the-art complementary prefetchers, Margo is able to achieve a 9.8% speedup in SPECspeed 2026 over a baseline equipped with a stride prefetcher. By contrast, Alecto [35], I-POP [37] and Perceptron Prefetch Filtering [8] achieve 1.2%, 7.6%, and 6.7% respectively, giving Margo a 2.1% speedup versus its closest competitor. Margo achieves this speedup while being highly DRAM efficient, increasing off-chip accesses by only 3.3% over a stride-only baseline, versus 8.7% for Alecto, 19.1% for I-POP, and 17.4% for PPF.

II. Motivation

A. Marginal Utility of Prefetchers

In economics, *marginal utility* refers to the additional benefit a consumer derives from consuming one additional unit of a commodity [64]. This concept implies that the utility of an additional unit of a product is inversely related to the number of units of that product that is already owned.

We observe that this concept of marginal utility also applies in scaling the number of data prefetchers, where the cumulative benefit of combining prefetchers is not strictly additive. Although data prefetchers are designed to exploit distinct and diverse data access patterns, their coverage frequently overlaps in practice. Two prefetchers, A and B, may both appear highly accurate in isolation. But if all of B's accurate prefetches are covered by A, but B adds additional inaccurate prefetches, then B adds no benefit while still consuming finite resources. If a prefetcher adds no additional value in the presence of other prefetchers but wastes cache and DRAM bandwidth through extra inaccurate prefetches, it could cause degradations to both overall system performance and energy efficiency.

In this paper, we use marginal utility as the primary metric for prefetcher management. Marginal utility captures not only the marginal accuracy of a prefetcher's requests, but also their timeliness. In particular, an otherwise less accurate prefetcher B may still provide additional benefit for prefetches already covered by a more accurate prefetcher A, provided that B is more timely. We refer this concept as Basic Marginal Utility (Section III-A). We further introduce Extended Marginal Utility (Section III-B), which augments the basic formulation with asymmetric weighting based on the source of each prefetch, following the Ignite Weighting principle of I-POP [37].

B. Input Filtering versus Output Filtering

Input and output filtering control different stages of prefetching. Input filtering determines which demand accesses train or consult a prefetcher, allowing it to focus its resources on selected access streams. Output filtering determines which generated candidates are issued, allowing the system to suppress ineffective requests while retaining the information used to learn access patterns.

Alecto [35] advocates input filtering by observing that different prefetchers target different memory access patterns. It samples the per-PC utility of each sub-prefetcher and dynamically directs incoming requests to an effective subset of prefetchers. However, applying such filtering requires care when a prefetcher learns across PCs: low utility for a PC does not necessarily imply that its accesses can be removed from training without affecting useful predictions for other PCs.

For example, consider Spatial Memory Streaming (SMS) prefetchers [5], [14], [21], [30], [58], [59], illustrated in Figure 2. The trigger PC that generates all prefetches for a region (for example, a 4k page) is almost always distinct from the PCs that the prefetches are intended to cover. The trigger merely marks the start of a spatial region, whereas the learned spatial footprint combines accesses from multiple instruction streams This discrepancy makes input filtering ineffective. If the generated prefetches are inaccurate, filtering the trigger PC does not eliminate them: subsequent accesses can associate the same faulty footprint with a different trigger, causing the inaccurate prefetches to be generated again (Figure 2, Case 1). Now consider the opposite case: the generated prefetches are consistently accurate, yet the prefetcher never produces a useful prefetch aimed at the trigger PC itself (for example, because it is fetching from a struct with irregular accesses outside the struct). Filtering that trigger suppresses the entire footprint and discards useful prefetches (Figure 2, Case 2).

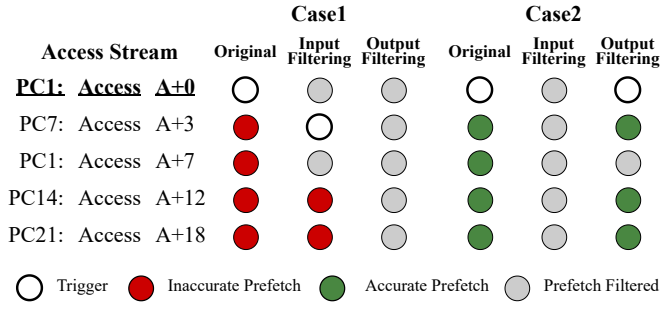


Fig. 2: Input versus output filtering for SMS-based prefetchers. In Case 1, filtering the trigger PC causes the region's subsequent accesses to associate with a different trigger PC7, and most inaccurate prefetches are still generated. In Case 2, input filtering suppresses all generated prefetches, including accurate ones. Output filtering instead removes inaccurate prefetches associated with PC1 while preserving useful ones.

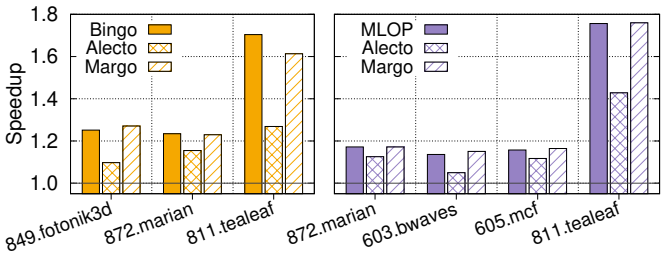


Fig. 3: Speedup of Bingo (left) and MLOP (right) with and without Alecto (input filtering) and Margo (output filtering). Both prefetchers rely heavily on cross-PC prefetching and suffer substantial slowdowns under input filtering, due to misattributed inaccurate prefetches. Output filtering largely preserves and improves the baseline performance.

Alecto thus introduces large performance penalties when applied to Bingo [5], an SMS-based prefetcher (Figure 3).

Global offset prefetchers (like BOP [39] and MLOP [55]) also learn across PCs. They derive prefetching offsets from a global memory access stream and issue these offsets on every PC. Therefore, harmful prefetches from global offset prefetchers would be issued on PCs that have no relation with the PC they're trained on. Hiding PCs that global offset prefetchers do poorly on not only does not improve their training, it also hinders them from discovering new prefetch candidates altogether. Again in Figure 3, MLOP sees significant slowdowns from Alecto.

For these prefetchers, output filtering can act on generated candidates without directly removing useful accesses from training.With sandbox-based [52] sampling, Margo can measure the effectiveness of each prefetcher in isolation, always observing the candidates it would issue without filtering, and select combinations without interference from feedback loops induced by currently issued prefetches.

By contrast, for space-constrained temporal prefetchers [2], [3], [18], [66], input filtering can complement output filtering by controlling which accesses consume scarce metadata capacity. Here, we can identify demands whose timely prediction is already provided by cheaper prefetchers: not based on accuracy, but on coverage instead, which can be measured at the PC-local level of the *target* even when prefetches are issued cross-PC (so the *source* may differ). When cheaper prefetchers can cover a demand in time, the temporal prefetcher should not spend its scarce metadata on redundant predictions. That capacity is better used for patterns that no other prefetcher can effectively prefetch, or in prefetchers that store their metadata in the LLC, reclaimed as additional cache space. In this case, we avoid training on loads that are *well covered* by other prefetchers (Section III-C), and instead train on loads that other prefetchers either fail to predict or predict too late for the prefetch to hide the latency.

Margo uses both mechanisms for different purposes: coverage-based input filtering manages the metadata capacity of space-constrained temporal prefetchers, while output filtering controls which generated candidates are issued. Neither mechanism fully disables a prefetcher in Margo.

III. Concepts

Here we first introduce the concept of *basic marginal utility*, which calculates the fraction of latency hidden by a prefetch of each prefetcher, sorted in terms of which is most accurate. We then define *extended marginal utility*, which adds in the relative cost-benefit of the prefetcher based on I-POP-style LLC and DRAM effects [37]. Finally, we develop *coverage utility*, which considers the fraction of loads that can be successfully covered by cheap prefetchers to avoid covering them by costly prefetchers such as Triangel [2].

A. Basic Marginal Utility

We define *Positive Basic Marginal Utility* (^{+}MU) in this work as the incremental memory-access latency that a prefetch uniquely hides with respect to other superior sources of prefetches targeting the same cache line. More specifically, this reflects the proportion of latency cycles that would have not been hidden had the prefetch not been issued. Consequently, a prefetch that is accurate but hides no additional latency in the presence of other more accurate prefetchers provides zero marginal utility. A useless prefetch (a prefetched line evicted from the cache before being accessed) provides no latency benefit while consuming cache and memory-system resources, and therefore incurs negative basic marginal utility (^{-}MU).

However, relying solely on basic marginal utility introduces a problematic feedback loop. When multiple prefetchers target the same cache line, they must be ranked to determine how the hidden latency should be attributed among them. If marginal utility itself is used for this ranking, a prefetcher already believed to have high marginal utility would receive credit more readily, disproportionately inflating their scores and reinforcing the initial belief. Such a ranking would suppress the evidence needed to correct itself. Hence, we rank each prefetcher by their *absolute accuracy* when attributing their share of latency hidden. Absolute accuracy is the proportion of a prefetcher's hypothetically prefetched lines that are accessed before eviction, including prefetches that would have been

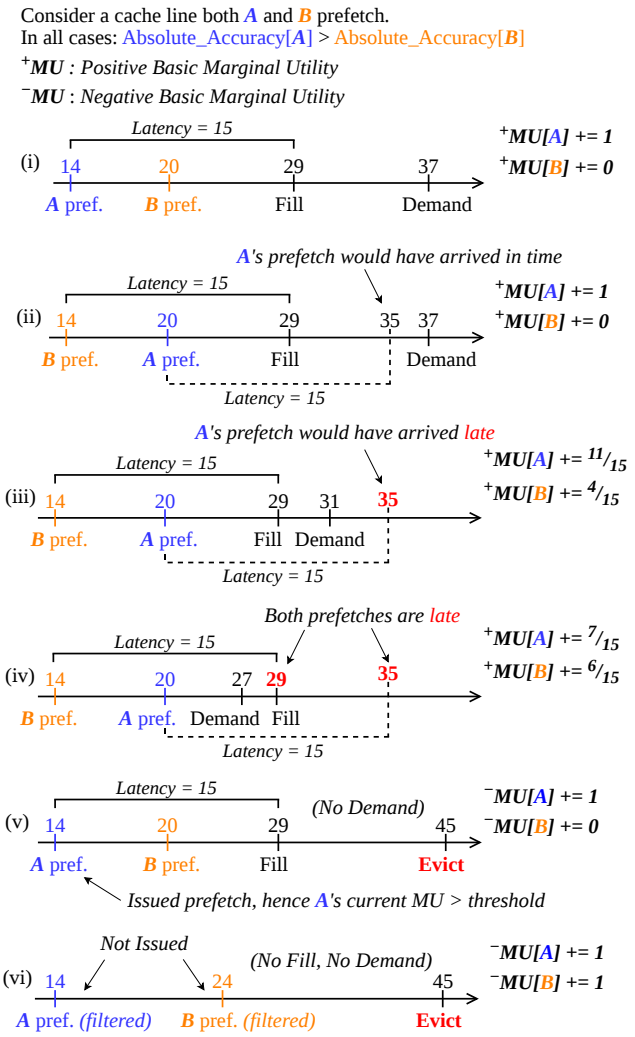


Fig. 4: Illustration of Basic Marginal Utility calculation in various circumstances.

issued had they not been filtered. Using absolute accuracy breaks the feedback loop since it is measured in a way that is masked from any of the filtering decisions derived from marginal utility. Both marginal utility and absolute accuracy are aggregated over the source PC (the PC of the access that triggered the prefetch) rather than the destination PC (the load PC that consumed the prefetched line). This accounts for cross-PC prefetching, so that marginal utility can be used at issue time to filter the prefetcher's decisions rather than to filter the training inputs (Section II-B).

Basic Marginal Utility can be considered an extension of *Marginal Accuracy*. Marginal accuracy is attributed to the most accurate prefetcher that covered an access but deducted for any prefetcher that issued an inaccurate prefetch. Basic marginal utility extends this by considering timeliness, where the *proportion* of latency hidden by each prefetcher is accounted for. This is then combined with Ignite Weighting (Section III-B) as Extended Marginal Utility (Section III-B).

Figure 4 provides some examples to calculate the attribution of Basic Marginal Utility. The examples shows two prefetchers, but it can be extended to an arbitrary number of prefetchers. Consider a cache line that is prefetched by two prefetchers A and B. In all examples, the absolute accuracy of A is larger than that of B, and the basic marginal utility is attributed to the PC that triggered the prefetch for A and B.

(i) A issues a prefetch for the cache line at cycle 14, while B issues a prefetch at cycle 20. The cache line is filled at cycle 29 due to A's prefetch and the demand for the line arrives at cycle 37. A was able to cover the entire fill latency without the presence of B. On the other hand, B is neither more timely nor more accurate than A. A receives all credit for covering the load and B receives none.

(ii) In this case, B issues the prefetch at cycle 14 and A at 20. The cache line has a fill latency of 15 cycles, fills at cycle 29 due to B's prefetch, and the demand arrives at cycle 37. Compared to the prior example, even when B successfully prefetched the line and is more timely than A, it does not bring any additional utility. Since if A were the only prefetcher, its prefetch would still arrive in time (cycle 35) before the demand (cycle 37). Hence, A receives all the latency credit for this load and B receives none.

(iii) Here, all the conditions are the same as (ii) except that the demand arrives on cycle 31 instead of 37. A would not be able to cover the full latency, while B was able to cover the full latency but not accurately. The latency credit is therefore split between A and B, where A gets attributed first ($+\frac{11}{15}$) and B gets the remaining credit ($+\frac{4}{15}$).

(iv) This presents a case where no prefetcher was able to cover the full latency of a load (late prefetch). Here prefetchers are awarded based on the portion of latency they managed to hide. Similar to (iii), we credit the more accurate prefetcher $A(+\frac{7}{15})$, then $B(+\frac{6}{15})$. If B were to prefetch the line at cycle 20 and A at cycle 14, similar to (ii), A would get partial credit ($+\frac{13}{15}$) and B none.

(v) In this instance, prefetcher A had sufficient marginal utility to issue a prefetch that was evicted before use (an inaccurate prefetch). Since A is the most accurate prefetcher and its marginal utility exceeds the threshold, it is currently still willing to issue the prefetch. Whether B redundantly prefetches the same address is irrelevant. Only A receives an increment to *negative* marginal utility.

(vi) Here, both prefetches from A and B were deemed inaccurate and filtered by Margo. This filtering decision was correct since both prefetches were inaccurate, so to reinforce this belief, both A and B receive an increment to *negative* marginal utility.

The positive marginal utility ${}^{+}MU$ and negative marginal utility ${}^{+}MU$ are interpreted as a cumulative sum of ${}^{+}MU$ and ${}^{-}MU$, to get a positive or negative marginal utility score which is used to drive prefetch filtering decisions.

B. Ignite Weighting and Extended Marginal Utility

We also borrow the concept of weighting prefetches based on the origin of their fill from I-POP [37] as *Ignite Weighting*. A prefetch filled from DRAM hides far more latency than one filled from the LLC, so it is worth more when it is accurate.

When it is inaccurate it is also more harmful, since it consumes scarce off-chip bandwidth and exacerbates contention.

Concretely, we evaluate the utility of prefetches asymmetrically by augmenting updates for basic marginal utility (Section III-A) by multiplicative factors, varying for both accurate and inaccurate LLC / DRAM prefetches. We call this *Extended Marginal Utility*, a utility metric that scales to a vast set of prefetchers with widely varying levels of aggression. This value is encoded as a probabilistic counter (Section IV-C1) in the prefetch filter. Positive and Negative Extended Marginal Utility (^{+}EMU and ^{-}EMU) at the prefetch-issue PC of prefetcher A are thus calculated as follows:

- Accurate prefetches:
  $^{+}EMU[A] = {}^{+}MU[A] \times (OnChip\ ?\ 3\ :\ 5)$
- Inaccurate prefetches:
  $^{-}EMU[A] = {}^{-}MU[A] \times (OnChip\ ?\ 1\ :\ 5)$

C. Coverage Utility

Temporal prefetchers (like Triangel [2]) face the issue of space constraint, where the issue is well-explored in recent literature [3], [18], [28]. We argue that in a multi-prefetcher setting, there is no need to inform the temporal prefetcher on demands that are already accurately and timely *covered* by other prefetchers (Section II-B). This frees up limited table capacity in the temporal prefetcher for complex patterns that no other prefetcher can capture.

We use a new metric, *coverage utility* (CU), to track the proportion of demand miss latency that covered by all non-temporal prefetchers at a given destination PC Y[1]. Different from marginal utility that tracks the *source* PC of a prefetch for *output filtering*, coverage utility is attributed to the *destination* PC that a prefetch targets. When a load at PC Y is partly or wholly covered by an accurate (not filtered out) non-temporal prefetch, the CU of PC Y is updated according to the fraction of latency covered and the current coverage utility at PC Y. If the fraction of latency covered is above the current CU, positive coverage utility is incremented (^{+}CU+=1), reflecting that this load has been more effectively covered than believed.

CU is represented as a ratio between positive and negative coverage utility $^{+}CU/(^{+}CU + {}^{-}CU)$ between 0.0 and 1.0. As CU approaches 1, we cover all the loads from this PC with other prefetchers, and the temporal prefetcher should not be informed of loads from this PC. This value is encoded in a stochastic counter (Section IV-C1).

IV. Margo

A. Organization of Margo

As shown in Figure 5, Margo consists of a Prefetch Sampler to gather statistics on prefetches and cache misses, a Prefetch Filter to filter sub-prefetcher outputs, and a Coverage Filter to filter the temporal sub-prefetcher's training inputs. Figure 6 shows the fields in each table, with the filters indexed by PC and the samplers indexed and tagged by load address.

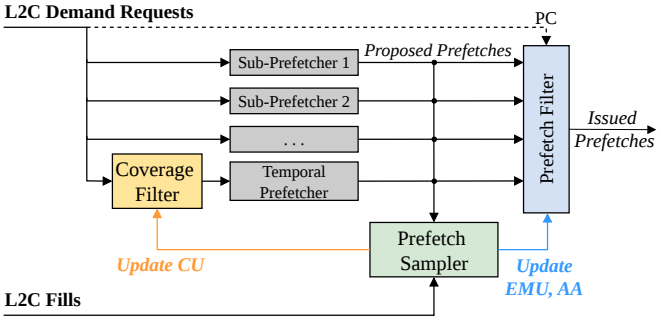


Fig. 5: Organization of Margo.

B. Prefetch Sampler

Margo relies on a light-weight Prefetch Sampler to gather the statistics needed to update Absolute Accuracy (AA), Extended Marginal Utility (EMU) and coverage utility (CU) per-PC. The Prefetch Sampler records prefetch candidates from all sub-prefetchers, regardless of whether they were issued or filtered away by Margo, as a form of sandbox prefetching [52]. This decouples a sub-prefetcher's evaluation from the outcome of previous evaluations, allowing Margo to observe the true behavior of each sub-prefetcher and avoid a feedback loop in which a suppressed sub-prefetcher never gains the evidence needed to recover.

In order to preserve the invariant that all attempts from all prefetchers to prefetch a given line are stored in the sampler, the decision to sample a prefetch or not is based purely on a function of address bits. This allows the samplers to observe all events belonging to a sampled line throughout its lifetime.

We dynamically calibrate the sampling rate of the prefetch sampler to faithfully and accurately measure whether a hypothetical prefetch would be used before eviction from the cache [2]. For sampled prefetches that are actually issued (not filtered)[2], the sampling rate is calibrated so that their eviction from the L2C and from the sampler are equally likely. This allows a single sampler to store metadata for the proposed prefetch candidates from sub-prefetchers in case they are used at any time before eviction, thus precisely measuring their accuracy and timing characteristics regardless of whether they were filtered or not. Retuning occurs when there are at least 400 evictions and there is a 2:1 imbalance between sampler and cache evictions. The sampling rate is then halved or doubled to reduce the imbalance, and the same process then repeats later if the imbalance persists.

A prefetch sampler entry is allocated or updated when any sub-prefetcher outputs a prefetch that falls within the sampled

[1]Correctly attributing coverage is non-trivial. Even when each cache line carries a separate tag in the cache to identify its prefetch source, the most aggressive prefetcher will set the tag, hence if the temporal prefetcher is the most aggressive, it claims coverage even when a cheaper, less aggressive prefetcher would have been just as timely. Similar to marginal utility, we solve this issue via sampling (Section IV-B), which allows us to attribute each prefetched line to the specific prefetcher(s) that issued it.

[2]To further achieve this, Margo also intentionally leaks a percentage (1%) of filtered prefetches into the system to calibrate the sampler with the behavior for otherwise-suppressed prefetches, to keep calibration maintained even if no prefetchers are currently accurate enough to issue prefetches.

address subset that would have been issued, independent of filtering or other prefetchers. Hence, the prefetch sampler also tracks prefetches for lines that are already in the cache or the MSHRs with the prefetch tag[3] being set (which is unset when a prefetch is first used). If the tag is unset, the prefetch is redundant and is ignored for the purposes of sampling, as it will be neither accurate nor inaccurate.

As shown in Figure 6, each Prefetch Sampler entry tracks the prefetch issue timestamp and source PC[4] of each sub-prefetcher that requested to prefetch this line, the timestamp of the cache fill for the line, and the timestamp of the demand that consumed the line[5], and a boolean flag indicating whether it was an on-chip memory access or not[6]. The fill latency can be derived by subtracting the earliest prefetch request timestamp from the line's cache fill timestamp[7]. The prefetch sampler returns an entry when the line reaches its terminal event: a demand access to the line (fill before demand, timely prefetch), a cache fill for the line (demand before fill, late prefetch), an eviction from the cache (useless prefetch), or a capacity eviction from the sampler (treated as a useless prefetch). When the entry is returned, it carries enough information to update the absolute accuracy, extended marginal utility, and coverage utility counters for its contributing PCs (Section IV-C1).

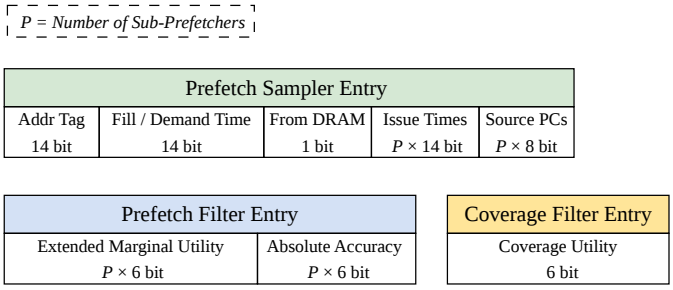


Fig. 6: Entry fields and sizing of Margo's structures.

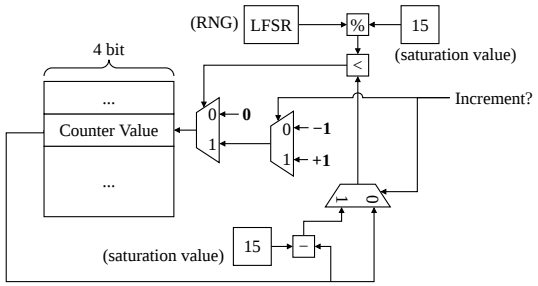


Fig. 7: Hardware realization of a 4 bit Stochastic Counter.

C. Filters

Margo utilizes two filters: the Coverage Filter to selectively filter the training inputs to the temporal prefetcher, and the Prefetch Filter to select which prefetchers should issue prefetches into the system.

1) Counters: To maintain the fractional metrics needed to make filtering decisions, Margo uses two types of counter that are *probabilistically* updated [53]. Margo's metrics are either expressed as fractions (e.g., cumulative fractional credit for marginal utility) or as ratios of cumulative metrics (e.g., absolute accuracy as useful prefetches / useless prefetches). To avoid storing large floating point numbers and expensive division logic, we use saturating counters that are probabilistically updated to record these metrics. This can be done with a cheap pseudo-random number generator via a linear-feedback shift register (LFSR) [10], a comparator, and two multiplexers.

To express ratios of cumulative metrics (as with absolute accuracy and coverage utility), we employ a *stochastic counter* (Fig. 7) whose update probabilities depend on its current value. Given a counter with maximum value M and current value n, an increment materializes with probability $(M-n)/M$ and a decrement with probability n/M. Examples are shown in Fig. 8. Each metric counter is incremented on positive events (useful sampled prefetches) and decremented on negative events (useless sampled prefetches for accuracy, demand misses within sampled sets for coverage utility). If positive events are drawn from a distribution with probability p, the counter increments with probability $p(M-n)/M$ and decrements with probability $(1-p)n/M$. As long as the underlying distribution does not change, n converges to pM given enough observations. This allows us to represent full ranges of probabilities between 0 and 1 to *compare* prefetchers, rather than saturating in either direction at high probability when a prefetcher is even slightly biased, otherwise following a random walk, as occurs with conventional saturating counters [42], [53].

To accumulate fractional values (as with marginal utility), we employ a *probabilistic counter* (Fig. 9) that is updated only according to a given input. To attribute a fractional positive Extended Marginal Utility (EMU) value, we increment the counter with the probability equal to its fractional EMU. Since negative EMU penalty is always a multiple of 1, the counter deterministically counts down for every decrement.

[3]A prefetch tag is a 1-bit flag kept with each cache line and MSHR entry, common to most prefetchers [20], [41], [57]. It is set when a prefetch fills the line and cleared on the first demand hit, to mark where the demand miss would have occurred were it not for the previous prefetch.

[4]A single prefetcher may attempt to prefetch a line at several different PCs. The first PC to attempt to issue a prefetch, regardless of whether it gets filtered away, is the one sampled. The one exception is when the first request is filtered and the same prefetcher later produces the request that is actually issued (i.e. no other prefetcher has claimed the line first, so the request misses in the cache and goes out as a new prefetch). In that case we overwrite the recorded PC with the later one, so that accuracy is attributed to the PC that actually performed the prefetch.

[5]The fill time is needed for timely prefetches, and the demand time is needed for late prefetches (where the demand arrives before the cache fill). Since these are mutually exclusive, both share a single timestamp field, and the entry's terminal event tells us which one it holds.

[6]For sampled entries containing only filtered-out prefetches, when terminating as an inaccurate prefetch, it carries no information about the fill / demand time or whether it would have been an LLC or DRAM access. Fill and demand times are only used for updating accurate prefetches, so these fields are not required. However, we still need to know the hypothetical origin of the prefetch to appropriately punish inaccurate DRAM traffic via Ignite Weighting (Section III-B). In this scenario, we check the LLC for the sampled line.

[7]For late prefetches where the demand time is stored in the sampler entry, the latency is calculated at fill time. This is calculated as fill time subtract the earliest of the stored demand time and the issue time.

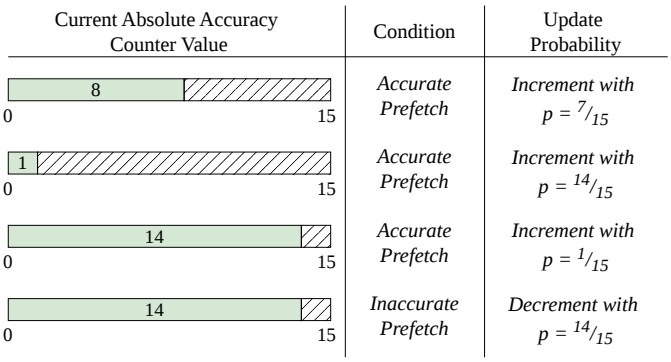


Fig. 8: A set of 4-bit stochastic counters representing absolute accuracy, and their probabilistic update response with respect to accurate and inaccurate prefetch candidates.

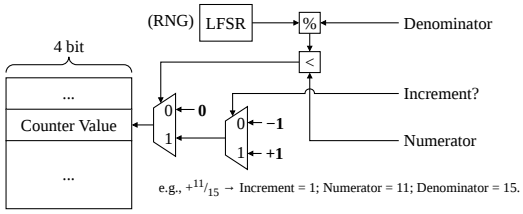


Fig. 9: Hardware realization of a Probabilistic Counter.

Given enough observations, the counters reflect whether the cumulated EMU is positive or negative.

2) Prefetch Filter: The Prefetch Filter is used to decide for each PC, whether a prefetcher should issue prefetches. As shown in Figure 6, each prefetch filter entry contains extended marginal utility (EMU) and absolute accuracy (AA) counters for each prefetcher. AA is used to calculate the EMU, while only EMU is used to make filtering decisions. Given a proposed prefetch candidate from a sub-prefetcher A at PC X, Margo allows it to be issued if $EMU[A]$ at X is positive (most significant bit 1).

3) Coverage Filter: The Coverage Filter is used to decide for each PC, whether a temporal prefetcher should be informed of its associated loads. As shown in Figure 6, each coverage filter entry holds a Coverage Utility (CU) (Section III-C) counter. Given a demand miss or tag hit at PC Y, Margo informs the temporal prefetcher of the demand if $1 - CU$ at Y is above a threshold of 30%. Examples are given in Fig. 10.

CU is implemented as a stochastic counter (Fig. 8) updated by misses and prefetches within the prefetch sampler's sampled sets. If there was a demand miss, the counter is stochastically decremented. If there was a prefetch and the address is in the sampler (whether a tag hit or an MSHR hit from a non-timely prefetch), the counter is stochastically incremented if the fraction of latency covered is higher than the counter's current value (interpreted as a fraction between 0 and 1) and decremented stochastically otherwise.

V. Experimental Setup

We evaluate Margo on the latest stable version of Champsim [26], simulating a processor with parameters listed in Table I. We compare Margo against the state-of-the-art prefetcher selection algorithms Alecto [35] and I-POP [37]. We also compare against PPF [8]. While PPF was originally proposed as a filter for the Signature Path Prefetcher [33], we extend the design to to filter candidates from arbitrary prefetchers by attaching a filter to each sub-prefetcher and replacing SPP-specific features with prefetcher-agnostic features. Their configuration and sizing is provided in Table III. Prefetchers are placed in the L2C for evaluation. Results are normalized to the IP-Stride [13] prefetcher, following previous work [47].

Table II gives the sizing of Margo's structures, totaling 2.0KiB with every managed prefetcher adding 1.8KiB of state, mainly due to more information being recorded in the sampler. All comparison schemes use the configuration and sizing as in their original proposals, as detailed in Table III. Margo is larger than I-POP and Alecto, but significantly smaller than Perceptron Prefetch Filtering (PPF) ($P\times$35KiB) and negligible compared with the size of modern L2Cs [9].

In total, we evaluate on 428 traces of simpoints from 4 benchmark suites: SPECspeed 2017 (95 traces) used in DPC3 [62], SPECspeed 2026 [61] (137 traces) generated in-house, Ligra (76 traces) open-sourced by Pythia [7], and

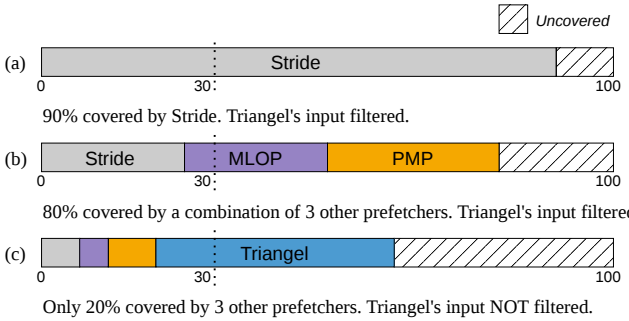
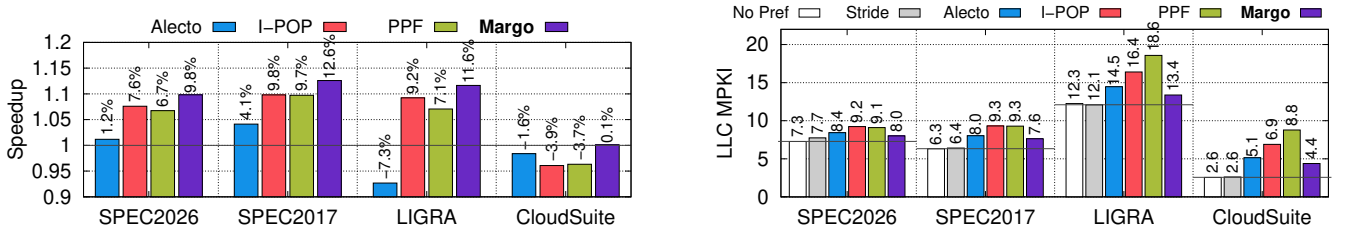


Fig. 10: An example of decision-making via coverage utility, using a 30% threshold and Triangel as the temporal prefetcher. Bars represent the proportion of latency coverable by each prefetcher. Triangel will be trained if this combined proportion from non-Triangel prefetchers is below a threshold for a given PC. Coverage utility is the aggregate coverage of all non-Triangel prefetchers, they are broken out here for illustration.

Core	Out-of-order, hashed perceptron branch predictor, 4 GHz with 6-issue width, 5-retire width, 352-entry ROB
TLBs	L1 iTLB & dTLB: 64 entries, 4-way, 1 cycle STLB: 1536 entries, 12-way, 8 cycles
L1I	32 KiB, 8-way, 4 cycles
L1D	48 KiB, 12-way, 5 cycles, LRU, non-inclusive
L2C	2 MiB, 16-way, 10 cycles, LRU, non-inclusive
LLC	6 MiB / core, 16-way, 20 cycles, LRU, non-inclusive
MSHRs	16/32/64 in L1D/L2C/LLC
PQs	8/16/32 in L1D/L2C/LLC
DRAM	16 GiB, 3200 MTPS, 1 channel, 8 bank groups, 4 banks per group, 8 KiB row-buffer per bank

TABLE I: Simulation parameters for baseline system.



(a) Geomean Speedup.

(b) Average Off-Chip DRAM Accesses (LLC MPKI, lower better).

Fig. 11: Headline performance results for single-core evaluation across various benchmark suites.

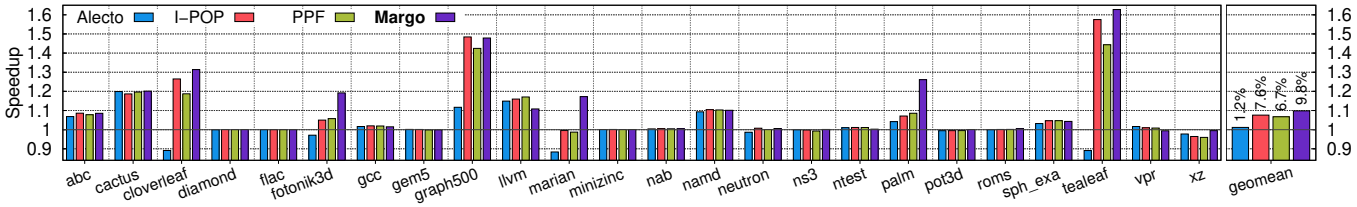


Fig. 12: Per-workload speedups on single-core SPECspeed 2026.

Structure	Size	Configuration	Storage
Prefetch Sampler	64×8	Addr Tag (14b) Fill/Demand Time (14b) From DRAM Flag (1b) Issue Times (P×14b) Source PCs (P×8b)	1.8KiB + P×1.4KiB
Prefetch Filter	256	Ext. Marginal Util. (P×6b) Absolute Acc. (P×6b)	P×384B
Coverage Filter	256	Coverage Utility (6b)	192B
Total	2.0KiB + P×1.8KiB		

TABLE II: Sizing of Margo's structures.

Alecto [35]	DB = 0.05, PB = 0.75, 64-entry allocation / sample table, 512-entry sandbox	0.6KiB + P × 0.22KiB
I-POP [37]	OFF_j, $j \in [1, 3]$, ON_i, $i \in [1, 5]$, degree $= 2 \cdot i$, 512-entry pfHT / PoHT	0.9KiB + P × 0.14KiB
PPF [8]	8 features, 4096-entry weights table, 1024-entry prefetch / reject filter	P × 35KiB

TABLE III: Configuration of comparison schemes.

CloudSuite (120 traces) used in CRC2 [63]. Each trace is warmed up for 50 million instructions before being timed for a 200 million instruction window. All presented results are aggregated based on the combined simpoints for each workload, and weighted whenever weights are available.

All evaluated prefetcher management schemes use the following prefetchers unless otherwise stated: IP-Stride [13], Next-Line [57], Berti [47], MLOP [55], Bingo [5], PMP [30], Pythia [7] and Triangel [2]. All sub-prefetchers use their proposed configurations or ChampSim defaults where relevant. Berti is adapted to train on L2 accesses, measures latencies in the L2, and prefetches into the L2 and LLC.

VI. Evaluation

A. Single-Core Summary

Figure 11 shows that Margo outperforms all contenders with both higher speedups and lower DRAM traffic (measured in LLC misses-per-kilo-instruction, or MPKI) in all evaluated suites. Margo outperforms I-POP, its closest competitor, on SPECspeed 2026, SPECspeed 2017 and Ligra, by 2.1%, 2.6% and 2.2% respectively, while generating far fewer off-chip accesses. On Cloudsuite, Margo outperforms its closest competitor Alecto by 1.7%, and its frugal DRAM-bandwidth usage makes it the only prefetch selector to avoid a slowdown.

In all suites, Alecto underperforms its contenders by filtering inputs based on the behavior of outputs (Section II-B). In Ligra, this causes it to significantly underperform the IP-stride baseline, by filtering away good prefetches. This is evidenced in Fig. 14, where Alecto has good accuracy but low coverage.

By contrast, I-POP and PPF generally sees bigger speedups than Alecto but also bigger slowdowns in workloads like Cloudsuite. While PPF is a powerful perceptron-based filter, it does not account for the marginal utility (Section III-A) between different prefetchers, causing it to issue redundant prefetches. I-POP is able to decisively throttle and switch off misbehaving prefetchers, but its effectiveness is limited by evaluating each prefetcher in isolation and making coarse-grained per-prefetcher decisions rather than filtering per-PC.

As we see in Figure 11b, Margo exhibits consistently low off-chip traffic. In all benchmarks, Margo's LLC MPKI remains close to baseline despite equipping eight prefetchers instead of one. Conversely, all other evaluated mechanisms induce significant traffic, with PPF more than tripling DRAM traffic in CloudSuite. Margo achieves high efficiency by evaluating the marginal utility of each sub-prefetcher and selecting only the most effective ones for a given access pattern.

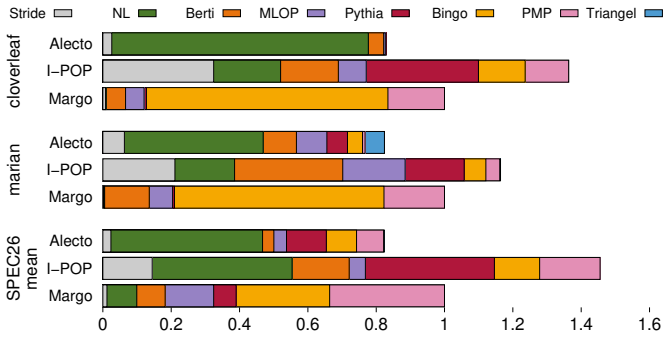


Fig. 13: Source breakdown of issued (non-redundant) prefetches by sub-prefetcher.

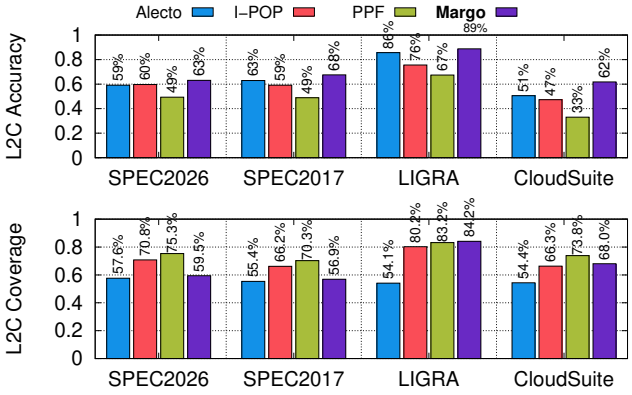


Fig. 14: Accuracy and Coverage in evaluated benchmarks.

While redundant requests from overlapping prefetches between multiple prefetchers in techniques like I-POP and PPF do not directly increase off-chip traffic (as they are caught by the cache or MSHRs), they end up inflating the observed accuracy of aggressive sub-prefetchers. This is because accuracy is attributed to the most aggressive prefetcher (which sets the tag) rather than the best prefetcher when they issue the same request, resulting in unnecessary traffic from prefetchers whose marginal utility is often poor. Alecto performs slightly better than I-POP and PPF, due to its dynamic allocation of individual requests to sub-prefetchers, but it still generates considerably more traffic than Margo. Part of this gap again stems from Alecto's non-marginal accuracy thresholds, but it also arises from how Alecto reassesses a blocked prefetcher. Because Alecto filters prefetcher inputs, the only way to gather the accuracy feedback needed to reclassify blocked prefetchers is to periodically re-enable them in full. In contrast, Margo reassesses prefetchers via virtualization within the samplers, issuing only a negligible fraction of a blocked prefetcher's requests for recalibration (Section IV-B).

B. SPECspeed 2026 Detailed Analysis

Figure 12 shows per-benchmark single-core speedup of evaluated prefetcher selection schemes in SPECspeed 2026. Margo is the only scheme that brings meaningful speedups in workloads like `fotonik3d`, `marian`, and `palm` whilst incurring the lowest LLC MPKI. `palm` interleaves many distinct stencil streams originating from distinct load PCs. Margo tackles this by fine-grained coordination of PMP (46%), MLOP (25%), and Bingo (22%) on the PCs they cover while suppressing Pythia and Berti elsewhere, resulting in 95% L2C coverage. While Alecto also makes PC-local decisions, the next-line prefetcher ends up dominating prefetching through being aggressive and yet accurate enough to pass Alecto's DB threshold. In `marian`, Margo sees aggressive filtering on all prefetchers, while still allowing prefetching on high-utility loads. As a result, it is the only mechanism to achieve a speedup, delivering a substantial 1.2× improvement. Other schemes fail in `marian` due to introducing excessive off-chip traffic, with Alecto, I-POP, and PPF seeing 1.36×, 1.35×, and 1.30× higher LLC MPKI than Margo. Notably, Margo's coverage filter prevents Triangel from training on `marian`, avoiding costly metadata storage in the LLC. In `fotonik3d`, Margo was brings the highest speedup with a combination of both high coverage (83.4%) and high accuracy (70%). This is due to Margo's ability to better coordinate between different prefetchers, using a mix of offset and regional prefetchers, while other schemes fall back to using simple stride prefetchers. Similarly, Margo avoids slowdowns in `xz` by preventing excessive off-chip DRAM traffic. One workload where Margo underperforms in is `llvm`, where the other three selection mechanisms increase performance at the expense of large traffic overheads (1.4-1.6×) that Margo inhibits.

C. Accuracy and Coverage

Figure 14 show prefetching Accuracy[8] and Coverage[9] in the L2C for each prefetcher management scheme across evaluated benchmarks. Despite all other evaluated schemes incorporating some measure of accuracy into their prefetcher selection metrics, Margo achieves the highest accuracy in all benchmark suites, precisely because it measures marginal benefit and disables prefetchers where no such benefit exists. Alecto is second best, since it disables prefetchers at a fine granularity, but its input filtering disables the wrong prefetches and leaves many inaccurate prefetches behind, explaining its low speedup despite often respectable coverage.

Margo sometimes exhibits lower coverage than other evaluated schemes, since it deliberately only issues prefetches when covering a load is consistently beneficial. Still, on individual memory-bound workloads, Margo often achieves higher coverage. Using `marian` and `fotonik3d` in SPECspeed 2026 as examples, Margo achieves 93% and 80% coverage respectively, significantly outperforming Alecto (65% and 49%), I-POP (82% and 60%), and PPF (85% and 73%).

Figure 13 shows the proportion of prefetches issued (accurate and inaccurate) on the whole of SPECspeed 2026 and two representative workloads by each sub-prefetcher for Alecto, I-POP and Margo, normalized to the number of prefetches issued by Margo. The distribution of prefetches from each sub-prefetcher is highly distinct between different selectors. Both

[8]Accuracy = Useful Prefetches / (Useful Prefetches + Useless Prefetches), where useless prefetches refer to prefetch requests unused before eviction.

[9]Coverage = Useful Prefetches / (Useful Prefetches + Cache Misses).

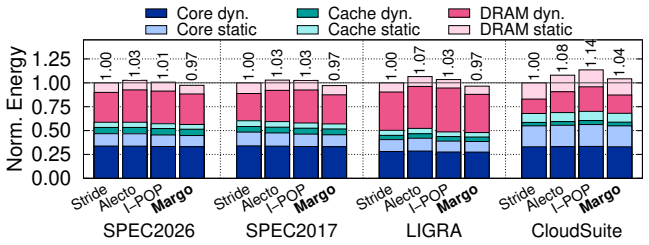


Fig. 15: Energy consumption in evaluated benchmarks.

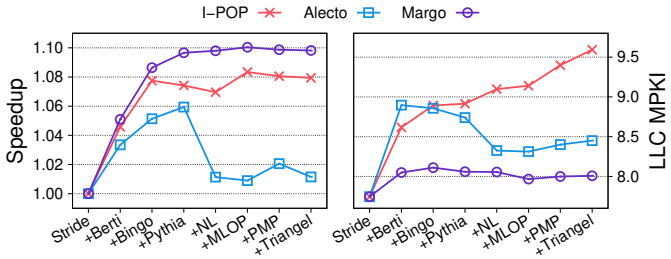


Fig. 16: Scalability by *adding more* prefetchers of Alecto, I-POP, and Margo on SPECspeed 2026.

Alecto and I-POP issue far more prefetches from NL than Margo does, because Margo realizes these prefetches, though often above 50% accuracy in isolation, add little marginal utility to other better candidates. By contrast, Bingo and PMP issue far more prefetches in Margo than elsewhere, due to Alecto incorrectly filtering cross-PC prefetches (Section II-B), and due to I-POP's coarse-grain filtering where their benefits are hidden by other more aggressive prefetchers. Margo uses every prefetcher (bar Triangel – though Triangel is used in the other benchmark suites more extensively) since they all provide marginal benefit in some circumstances.

D. Energy Consumption

Figure 15 shows energy consumption of Margo, Alecto, and I-POP in the evaluated benchmarks normalized to a Stride prefetcher. We model core energy with McPAT [36] and cache energy with CACTI [6]. DRAM energy uses DRAMPower's model [11], [12] with Micron DDR4 currents [40]. In all suites, Margo exhibits the lowest energy consumption compared to I-POP and Alecto. Margo's high performance and low traffic translate to low static and dynamic power respectively, giving lower energy consumption than the stride prefetcher across all suites except Cloudsuite.

E. Multi-Prefetcher Scalability

As shown in Figure 16, Margo exhibits superior scalability as the the number of managed prefetchers increases. On SPECspeed 2026, Margo consistently exhibits higher speedups than Alecto and I-POP at any number of prefetchers. More notably, Margo achieves this while exhibiting constant or even lower LLC MPKI as more prefetchers are added, by virtue of its fine grain marginal evaluation of added prefetchers. I-POP exhibits a marked increase in off-chip accesses with each additional prefetcher, gaining little to no performance benefit

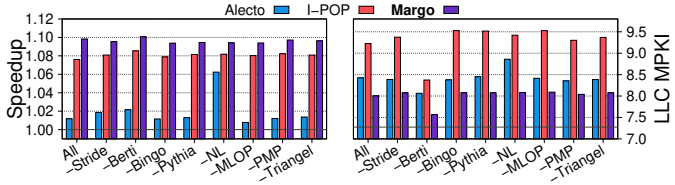


Fig. 17: *Disabling* individual sub-prefetchers (SPEC2026).

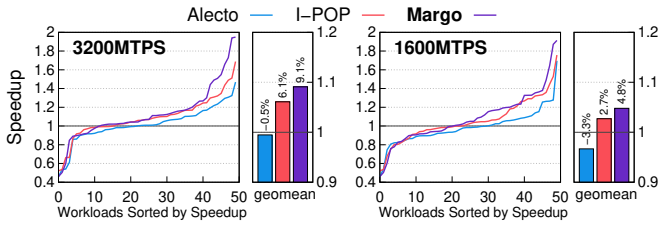


Fig. 18: Speedup on 8 core heterogeneous workloads.

in return. While Alecto fares slightly better than I-POP in managing off-chip traffic, it sacrifices performance and still incurs a higher LLC MPKI than Margo. Alecto especially suffers from NL's aggressiveness letting it claim attribution for most prefetches, depriving the other sub-prefetchers of credit and leaving them falsely throttled. For example, Berti, a highly accurate prefetcher [47], is falsely predicted to have only 32% average accuracy in the presence of NL by Alecto's sandbox.

Figure 17 shows the opposite study, by *removing* each individual sub-prefetcher from the ensemble. Margo and I-POP's performance stay largely unchanged, with Margo always the best performer. Many individual prefetchers have negative marginal utility for Alecto and I-POP, significantly harming both speedup and/or off-chip traffic, particularly NL for Alecto. Despite incurring a significant benefit when adding Berti (Fig. 16), we see that with all other prefetchers enabled, Berti slightly reduces the overall speedup even for Margo. Berti also incurs a large off-chip traffic penalty, since disabling it from Margo reduces traffic below even that of the baseline stride prefetcher (7.6 vs. 7.7 LLC MPKI).

F. Multicore Performance

Figure 18 compares Alecto, I-POP, and Margo across 50 randomly selected heterogeneous 8-core workloads at full and half DRAM bandwidth. In multicore and DRAM constrained scenarios, increased contention amplifies the cost of low-utility prefetches. Margo's consideration of marginal utility allows it to prioritize prefetches that actually hide additional latency. At full and half DRAM bandwidth, Margo outperforms I-POP by 2.8% and 2.0%, and Alecto by 9.7% and 8.4%, respectively. In both scenarios, Margo brings a higher performance ceiling compared to I-POP due to allowing accurate and aggressive prefetching when it matters. Alecto's accuracy-based input filtering makes it both timid and unable to consistently remove poor prefetches, as evidenced by its negative speedups.

G. Ablation Study

Figure 19 shows an ablation study of marginal utility, starting with PPF since it is also a fine-grained output fil-

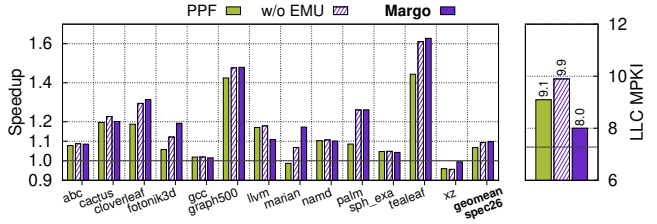


Fig. 19: Ablation study of Extended Marginal Utility on SPECspeed 2026, breaking down speedup for workloads with performance delta of >1%.

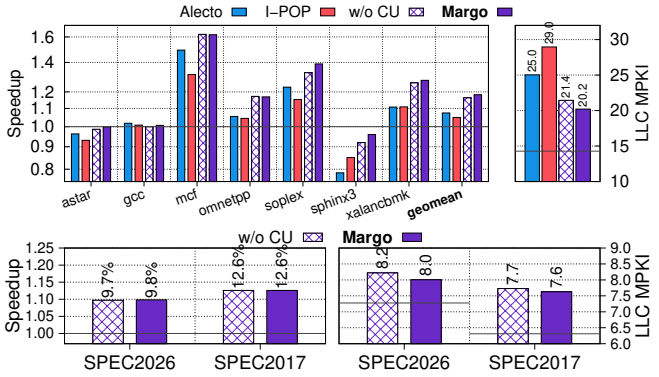


Fig. 20: Ablation of Coverage Utility on irregular SPEC CPU2006 workloads (as evaluated in Triangel [2]) as well as in SPECspeed 2026 and SPECspeed 2017.

ter. Margo with a filter based on absolute accuracy alone considerably outperforms PPF, in part since PPF's perceptron features are not well suited to generic prefetcher selection, but also because Margo's sampling and counter mechanisms provides an impartial view of all sub-prefetchers. However, without analysis of the marginal benefit between prefetchers (Section III-A), Margo generates untenable DRAM traffic that exceeds both PPF and I-POP. Similar to other schemes (Figure 12), this increased aggression helps in `llvm` but increases off-chip accesses by 62%. Marginal utility is vital for achieving speedups in `fotonik3d` and `marian`, and preventing slowdown in `xz`.

Figure 20 shows an ablation study of coverage utility, on the original benchmarks evaluated in Triangel [2]. Coverage utility improves Margo's performance by 4.3% on `soplex` and 4.5% on `sphinx3` by improving storage efficiency, eliminating redundant prefetches that cheaper prefetchers already cover within Triangel's finite structures. This is also accompanied by a significant drop in DRAM traffic, most evident in `sphinx3` where coverage utility was able to lower off-chip DRAM accesses by 47%. This is because coverage utility enables Triangel to reduce metadata storage, shrinking its partition sizes in the LLC. This is consistent with the trend in SPECspeed 2026 and 2017, where Coverage Utility improves DRAM traffic without harming performance. In SPECspeed 2026, coverage utility removes 26.7% of the excess DRAM traffic by allowing Margo to avoid allocating any LLC space

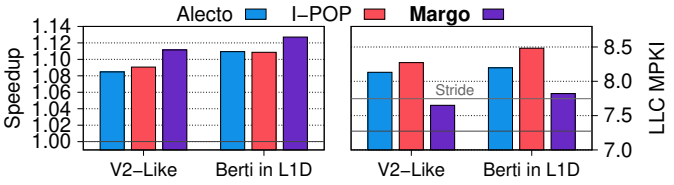


Fig. 21: Alternative L1 / L2 sub-prefetcher configurations for Margo, Alecto, and I-POP on SPECspeed 2026.

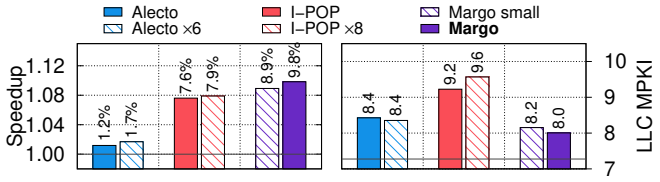


Fig. 22: Iso-storage study vs. Alecto/I-POP (SPEC2026).

for Triangel metadata storage in all workloads except `abc`[10].

H. Alternative Prefetcher Configurations

Figure 21 presents an ARM Neoverse V2 [9]-like prefetcher setup, with Margo, Alecto and I-POP managing Stride + PMP in the L1D and BOP [39] + Bingo + Triangel in the L2. Margo outperforms Alecto and I-POP by 2.5% and 1.9% respectively while incurring an even lower LLC MPKI than a Stride prefetcher running in the L2. As previously seen in Figure 17, Berti does not perform well in a large ensemble of L2 prefetchers, despite performing well on its own in the L2. Figure 21 shows Berti placed in its original location [47], the L1D, with the other seven prefetchers subject to Margo/I-POP/Alecto and placed in the L2. Again, Margo achieves the highest speedup with very low traffic.

I. Iso-Storage Comparison

Figure 22 shows an iso-storage evaluation of Margo, Alecto, and I-POP in SPECspeed 2026. Scaling Alecto's structures (AT, ST, and Sandbox) by 6× and I-POP's structures (pfHT and poHT) by 8× to match Margo yields only 0.5% and 0.3% additional speedup. By contrast, scaling down Margo (128-entry FT, 4-bit counters, 4-way 8-set sampler) preserves its performance, outperforming Alecto and I-POP by 7.6% and 1.2% respectively while still incurring the lowest LLC MPKI.

J. Sensitivity Analysis

Figure 23 shows the sensitivity of Margo with DRAM bandwidth, sampler sizing and Ignite Weights (Section III-B). Margo is able to outperform Alecto and I-POP in all DRAM bandwidth settings due to its frugal use of DRAM traffic. Margo achieves its highest speedup of 10.8% for a DRAM bandwidth of 6400 MT/s, maintaining its performance lead versus I-POP by 2.1% and Alecto by 8.8%. Margo is only

[10]In Ligra and Cloudsuite, Triangel already turns itself off due its sampling data indicating no reliable access pattern to capture. Other more aggressive prefetchers such as Triage [66] may benefit more from using Coverage Utility.

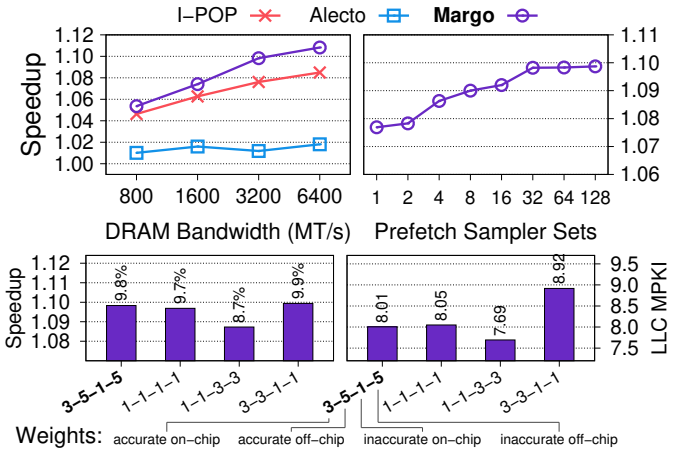


Fig. 23: Sensitivity of Margo under varying DRAM bandwidth, sampler sizes, and Ignite Weights on SPECspeed 2026.

weakly affected by the sizing of its prefetch sampler (Section IV-B) because it is actively calibrated to L2C capacity. Speedup saturates beyond 32 sets (256 entries): 9.82% versus 9.83% at the 64-set default, rising only to 9.87% at 128 sets, while shrinking to 2 sets lowers speedup to 7.83%. Following the principles of I-POP [37] via Ignite Weighting gives a slight extra speedup and slightly less DRAM traffic. Margo's performance is robust to the tuning of Ignite Weights, but it can be tuned to reduce bandwidth further when needed.

VII. Related Work

Prefetcher selection is a widely researched topic. Alecto [35] filters demand requests for prefetchers at a per-PC granularity at PCs that generate inaccurate prefetches. I-POP [37] uses more coarse-grained degree and on/off control based around the latency hidden by a prefetch and the cost of fetching it from cache or main memory. Earlier techniques limit their potential gains by isolating prefetcher operations: Division of Labour [34] trains only a single prefetcher per observation, whereas IPCP [50] accepts predictions from only one. ML-based Selector [4] and Micro-Armed Bandit [23] use reinforcement learning to control coarse-grained degree and on/off characteristics similar to I-POP. ReSemble [70] explores reinforcement learning over a wider parameter space. However, I-POP [37] demonstrates that reinforcement-learning strategies adapt slowly because they see limited information beyond an aggregate reward, limiting potential gains. Sandbox [52] measures hypothetical prefetches that were not issued, similar to Margo and PPF [8] – but does not measure their marginal accuracy or utility, and has a simple action space of choosing the top N prefetchers. PPF [8] is not natively a prefetcher selection algorithm, but represents a powerful mechanism for prefetchers operating in isolation, by predicting prefetch usefulness across many features. While we demonstrate Margo solely on PC-based features, we anticipate that its extension to perceptrons is simple albeit at increased storage.

Prefetcher control for individual prefetchers has also been explored in the literature. Feedback-directed prefetching [60] allows a prefetcher to change its aggression state internally and in isolation, while Margo adapts the global interactions between prefetchers (including where Margo improves the accuracy and contention metrics that feedback-directed prefetching uses as input). Hierarchical Prefetcher Aggressiveness Control [19] adds in varying thresholds based on LLC and main-memory contention. Margo can be used in conjunction with these techniques on individual prefetchers, such as by altering prefetcher degrees and Margo's weights (Section III-B) and thresholds. We see integration of degree feedback control at a global multi-prefetcher level, taking into account marginal utility, as important future work.

Other prefetch utility metrics include criticality [32], [38], [51], which we expect can be combined with extended marginal utility in future work. Alternative forms of marginal utility in data prefetching have also been explored in a different context, to handle the problem of Cache Pollution. Both Sequential Prefetching in Adaptive Replacement Cache [25] and Adaptive Multi-stream Prefetching [24] model the marginal utility of additional prefetches versus additional cache hit rates.

Margo and all prefetcher selection techniques rely on a rich body of underlying prefetcher techniques to cover various patterns. Stride prefetchers look for repeating address deltas at adjacent accesses at a per-PC granularity [13]. Offset prefetchers [16], [27], [39], [46], [47], [55] generalize to deltas from non-adjacent accesses. Region prefetchers [5], [14], [21], [30], [58] such as Spatial Memory Streaming [59], look for patterns that repeat across different memory regions (e.g. different 4k pages), for example due to repeated struct accesses. Temporal prefetchers [31], [48], [65] store address pairs, either in dedicated tables, cache partitions [2], [3], [18], [66], or off-chip [29], [67]. Signature Path Prefetcher [33] learns confidence-based sequences across PCs and accesses. Neural-inspired techniques [17], [56] and reinforcement learning [7] can also be used to produce correlations. Beyond address correlation, many prefetchers also look at data inside lines. Content-Directed Prefetching [15] observes loaded data and looks for integer values in the range of the heap, and fetches them as pointers. Indirect Memory Prefetchers [22], [69] look for data values used as indices with reference to base arrays. Runahead techniques [43]–[45], [54] replay the instruction stream to extract prefetches.

VIII. Conclusion

We have introduced Margo, a prefetch selection algorithm capable of handling the large number of prefetchers in both today's and tomorrow's processors. Margo makes use of prefetchers only when they improve system behavior, via its marginal utility metrics – and in doing so, manages significant performance improvement with low enough traffic for real-world, multicore deployment. We believe marginal utility can be deployed and extended in many further contexts – taking into account broader system behavior, broader parameters and metrics. We hope it is the stepping stone for a new generation of selection algorithms, and new complementary prefetchers that can be unleashed wherever they provide real benefit.

REFERENCES

[1] Advanced Micro Devices, Inc., "Software optimization guide for the AMD Zen5 microarchitecture," Advanced Micro Devices, Inc., Software Optimization Guide, Aug. 2024, publication No. 58455, Revision 1.00. [Online]. Available: https://docs.amd.com/v/u/en-US/58455_1.00

[2] S. Ainsworth and L. Mukhanov, "Triangel: A high-performance, accurate, timely on-chip temporal prefetcher," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024.

[3] S. Ainsworth and L. Mukhanov, "Tetrahedrangel: An energy-efficient coalescing temporal prefetcher," *ACM Trans. Archit. Code Optim.*, vol. 23, no. 3, Jul. 2026.

[4] E. S. Alcorta, M. Madhav, S. Tetrick, N. J. Yadwadkar, and A. Gerstlauer, "Lightweight ml-based runtime prefetcher selection on many-core platforms," 2023. [Online]. Available: https://arxiv.org/abs/2307.08635

[5] M. Bakhshalipour, M. Shakerinava, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Bingo: Spatial Data Prefetcher Repository," https://github.com/bakhshalipour/Bingo, 2019, gitHub repository.

[6] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.

[7] R. Bera, K. Kanellopoulos, A. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, "Pythia: A customizable hardware prefetching framework using online reinforcement learning," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21, 2021.

[8] E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez, "Perceptron-based prefetch filtering," in *Proceedings of the 46th International Symposium on Computer Architecture*, ser. ISCA '19, 2019.

[9] M. Bruce, "Arm Neoverse V2 platform: Leadership Performance and Power Efficiency for Next-Generation Cloud Computing, ML and HPC Workloads," in *2023 IEEE Hot Chips 35 Symposium (HCS)*, Aug. 2023.

[10] A. A. Bruen, M. A. Forcinito, and J. M. McQuillan, "Shift registers (lfsr) and stream ciphers," in *Cryptography, Information Theory, and Error-Correction: A Handbook for the 21st Century*, 2021, pp. 333–353.

[11] K. Chandrasekar, B. Akesson, and K. Goossens, "Improved power modeling of DDR SDRAMs," in *Proceedings of the 14th Euromicro Conference on Digital System Design (DSD)*, 2011, pp. 99–108.

[12] K. Chandrasekar, C. Weis, Y. Li, S. Goossens, M. Jung, O. Naji, B. Akesson, N. Wehn, and K. Goossens, "DRAMPower: Open-source DRAM power & energy estimation tool," https://github.com/tukl-msd/DRAMPower.

[13] T.-F. Chen and J.-L. Baer, "Effective hardware-based data prefetching for high-performance processors," *IEEE Transactions on Computers*, vol. 44, no. 5, 1995.

[14] Z. Chen, C. Wu, Y. Gu, R. Jia, J. Li, and M. Guo, "Gaze into the pattern: Characterizing spatial patterns with internal temporal correlations for hardware prefetching," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Mar. 2025.

[15] R. Cooksey, S. Jourdan, and D. Grunwald, "A stateless, content-directed data prefetching mechanism," *SIGPLAN Not.*, vol. 37, no. 10, p. 279–290, Oct. 2002.

[16] Y. Cui, W. Chen, X. Cheng, and J. Yi, "Hyperion: A highly effective page and pc based delta prefetcher," *ACM Trans. Archit. Code Optim.*, vol. 21, no. 4, Nov. 2024.

[17] Q. Dong, A. Jain, and C. Lim, "A new formulation of neural data prefetching," in *Proceedings of the 51st Annual International Symposium on Computer Architecture*, ser. ISCA '24, 2024.

[18] Q. Duong and C. Lin, "Streamlined on-chip temporal prefetching," in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2026.

[19] E. Ebrahimi, O. Mutlu, C. J. Lee, and Y. N. Patt, "Coordinated control of multiple prefetchers in multi-core systems," in *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO 42, 2009.

[20] B. Falsafi and T. F. Wenisch, *A Primer on Hardware Prefetching*, ser. Synthesis Lectures on Computer Architecture. Cham, Switzerland: Springer Cham, 2014. [Online]. Available: https://doi.org/10.1007/978-3-031-01743-8

[21] M. Ferdman, S. Somogyi, and B. Falsafi, "Spatial memory streaming with rotated patterns," *Journal of Instruction-Level Parallelism (JILP)*, vol. 13, 2011.

[22] G. Fu, T. Xia, Z. Luo, R. Chen, W. Zhao, and P. Ren, "Differential-matching prefetcher for indirect memory access," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.

[23] G. Gerogiannis and J. Torrellas, "Micro-armed bandit: Lightweight & reusable reinforcement learning for microarchitecture decision-making," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23, 2023.

[24] B. S. Gill and L. A. D. Bathen, "Optimal multistream sequential prefetching in a shared cache," *ACM Trans. Storage*, vol. 3, no. 3, p. 10–es, Oct. 2007.

[25] B. S. Gill and D. S. Modha, "Sarc: sequential prefetching in adaptive replacement cache," in *Proceedings of the Annual Conference on USENIX Annual Technical Conference*, ser. ATEC '05, 2005, p. 33.

[26] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. Pugsley, and J. Kim, "The championship simulator: Architectural simulation for education and competition," 2022. [Online]. Available: https://arxiv.org/abs/2210.14324

[27] S. He, Z. Wang, X. Tang, Q. Sun, and D. Dong, "Chimera: Leveraging Hybrid Offsets for Efficient Data Prefetching," in *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*, 2024.

[28] S. He, Z. Wang, X. Tang, H. Tang, D. Dong, and L. Xiao, "Elevating temporal prefetching through instruction correlation," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO, 2025.

[29] A. Jain and C. Lin, "Linearizing irregular memory accesses for improved correlated prefetching," in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO 46, 2013.

[30] S. Jiang, Q. Yang, and Y. Ci, "Merging similar patterns for hardware prefetching," in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '22, 2023.

[31] D. Joseph and D. Grunwald, "Prefetching using markov predictors," in *Proceedings of the 24th Annual International Symposium on Computer Architecture*, ser. ISCA '97, 1997.

[32] N. S. Kalani and B. Panda, "Instruction criticality based energy-efficient hardware data prefetching," *IEEE Computer Architecture Letters*, vol. 20, no. 2, 2021.

[33] J. Kim, S. H. Pugsley, P. V. Gratz, A. L. N. Reddy, C. Wilkerson, and Z. Chishti, "Path confidence based lookahead prefetching," in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-49, 2016.

[34] S. Kondguli and M. Huang, "Division of labor: a more effective approach to prefetching," in *Proceedings of the 45th Annual International Symposium on Computer Architecture*, ser. ISCA '18, 2018.

[35] M. Li, Q. Zhang, Y. Ren, and Z. Xie, "Integrating prefetcher selection with dynamic request allocation improves prefetching efficiency," in *Proceedings of the 31st IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.

[36] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "Mcpat: An integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proceedings of the 42nd annual ieee/acm international symposium on microarchitecture*, 2009, pp. 469–480.

[37] Y. Lin, W. Lin, Y. Chen, J. Xu, S. Cai, J. Ye, Z. Wang, and W. Chen, "I-POP: Ignite positive prefetchers," in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb. 2026, pp. 1–16.

[38] R. Manikantan and R. Govindarajan, "Focused prefetching: performance oriented prefetching based on commit stalls," in *Proceedings of the 22nd Annual International Conference on Supercomputing*, ser. ICS '08, 2008.

[39] P. Michaud, "Best-offset hardware prefetching," in *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2016, pp. 469–480.

[40] Micron Technology, Inc., "Calculating memory power for DDR4 SDRAM," Micron Technology, Inc., Technical Note TN-40-07, 2017.

[41] S. Mittal, "A survey of recent prefetching techniques for processor caches," *ACM Comput. Surv.*, vol. 49, no. 2, aug 2016.

[42] K. H. Mose. (2026, Jan.) Interpreting saturating counters in predictors. Mechanical Genie (Substack), accessed 2026-07-28. [Online]. Available: https://mechanicalgenie.substack.com/p/saturating-counters-in-predictors

[43] O. Mutlu, J. Stark, C. Wilkerson, and Y. N. Patt, "Runahead execution: An alternative to very large instruction windows for out-of-order processors," in *Proceedings of the 9th International Symposium on High-Performance Computer Architecture*, ser. HPCA '03, 2003.

[44] A. Naithani, S. Ainsworth, T. M. Jones, and L. Eeckhout, "Vector runahead," in *Proceedings of the 48th Annual International Symposium on Computer Architecture*, ser. ISCA '21, 2021, p. 195–208.

[45] A. Naithani, J. Roelandts, S. Ainsworth, T. M. Jones, and L. Eeckhout, "Decoupled vector runahead," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23, 2023.

[46] A. Navarro-Torres, B. Panda, J. Alastruey-Benedé, P. Ibáñez, V. Viñals Yúfera, and A. Ros, "A complexity-effective local delta prefetcher," *IEEE Trans. Comput.*, vol. 74, no. 5, Jan. 2025.

[47] A. Navarro-Torres, B. Panda, J. Alastruey-Benedé, P. Ibáñez, V. Viñals-Yúfera, and A. Ros, "Berti: an accurate local-delta data prefetcher," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022.

[48] K. Nesbit and J. Smith, "Data cache prefetching using a global history buffer," in *10th International Symposium on High Performance Computer Architecture*, ser. HPCA 10, 2004.

[49] NVIDIA Corporation, "NVIDIA Vera CPU architecture whitepaper," NVIDIA Corporation, Tech. Rep., Jul. 2026. [Online]. Available: https://nvdam.widen.net/s/nmw5vblpqd/nvidia_vera_cpu_architecture_whitepaper

[50] S. Pakalapati and B. Panda, "Bouquet of instruction pointers: Instruction pointer classifier-based spatial hardware prefetching," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.

[51] B. Panda, "CLIP: Load criticality based data prefetching for bandwidth-constrained many-core systems," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23, 2023, p. 714–727.

[52] S. H. Pugsley, Z. Chishti, C. Wilkerson, P.-f. Chuang, R. L. Scott, A. Jaleel, S.-L. Lu, K. Chow, and R. Balasubramonian, "Sandbox prefetching: Safe run-time evaluation of aggressive prefetchers," in *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*, 2014.

[53] N. Riley and C. Zilles, "Probabilistic counter updates for predictor hysteresis and stratification," in *The Twelfth International Symposium on High-Performance Computer Architecture, 2006.*, 2006.

[54] J. Roelandts, A. Naithani, S. Ainsworth, T. M. Jones, and L. Eeckhout, "Scalar vector runahead," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.

[55] M. Shakerinava, M. Bakhshalipour, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Multi-lookahead offset prefetching," *The Third Data Prefetching Championship*, no. 2019, 2019.

[56] Z. Shi, A. Jain, K. Swersky, M. Hashemi, P. Ranganathan, and C. Lin, "A hierarchical neural model of data prefetching," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21, 2021.

[57] A. J. Smith, "Sequential program prefetching in memory hierarchies," *Computer*, vol. 11, no. 12, 1978.

[58] S. Somogyi, T. Wenisch, M. Ferdman, and B. Falsafi, "Spatial memory streaming," *Journal of Instruction-Level Parallelism (JILP)*, vol. 13, 2011.

[59] S. Somogyi, T. F. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos, "Spatial memory streaming," in *Proceedings of the 33rd Annual International Symposium on Computer Architecture*, ser. ISCA '06, 2006.

[60] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, 2007.

[61] Standard Performance Evaluation Corporation (SPEC), "SPEC CPU 2026 benchmark suite," https://www.spec.org/cpu2026/, 2026, version 1.0, released May 2026.

[62] "Champsim traces for spec cpu," https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/, Stony Brook University, COMPAS / PACE Lab, 2025, accessed: 2025-11-15.

[63] The 2nd Cache Replacement Championship Organizing Committee, "The 2nd cache replacement championship trace suite," http://bit.ly/2t2nkUj, 2017, accessed: 2025-11-15.

[64] H. R. Varian, *Intermediate Microeconomics: A Modern Approach*, 9th ed. W. W. Norton, 2014.

[65] T. F. Wenisch, S. Somogyi, N. Hardavellas, J. Kim, A. Ailamaki, and B. Falsafi, "Temporal streaming of shared memory," in *Proceedings of the 32nd Annual International Symposium on Computer Architecture*, ser. ISCA '05, 2005.

[66] H. Wu, K. Nathella, J. Pusdesris, D. Sunwoo, A. Jain, and C. Lin, "Temporal prefetching without the off-chip metadata," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO 52, 2019.

[67] H. Wu, K. Nathella, D. Sunwoo, A. Jain, and C. Lin, "Efficient metadata management for irregular data prefetching," in *Proceedings of the 46th International Symposium on Computer Architecture*, ser. ISCA '19, 2019.

[68] C. Xu, X. Sun, W. Li, C. Bai, B. Wang, M. Li, Z. Xie, and Y. Xie, "Pf-llm: Large language model hinted hardware prefetching," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '26, 2026.

[69] X. Yu, C. J. Hughes, N. Satish, and S. Devadas, "IMP: indirect memory prefetcher," in *Proceedings of the 48th International Symposium on Microarchitecture*, ser. MICRO-48, 2015.

[70] P. Zhang, R. Kannan, A. Srivastava, A. V. Nori, and V. K. Prasanna, "ReSemble: reinforced ensemble framework for data prefetching," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, ser. SC '22, 2022.